Food Truck Live

Manual de instalação

GitHub, PostgreSQL, Supabase, Brevo, Vercel, Cloudflare e alojamento Node.js/Docker.

Guia ilustrado | 3 de novembro de 2026



Índice

1. Escolher o percurso certo 3
2. Compatibilidade e limites 4
3. Conteúdo do pacote e dados privados 5
4. Obter o código por GitHub ou ZIP 6
5. Variáveis de ambiente 7
6. Criar PostgreSQL no Supabase 9
7. Outro fornecedor PostgreSQL 10
8. Migrações e primeiro administrador 11
9. Armazenamento das imagens 12
10. Ativar email com Brevo ou SMTP 13
11. Percurso A: API no Vercel 15
12. Percurso A: sites no Vercel 16
13. Percurso B: servidor Docker 18
14. Cloudflare: DNS e proteção do Docker 19
15. Percurso C: clientes em Workers 20
16. Workers: gestão e GitHub 21
17. R2 para imagens, mantendo PostgreSQL 22
18. Percurso D: Node.js genérico 23
19. Gratuito ou pago 24
20. Testar antes de abrir ao público 25
21. Backup e restauro 26
22. Migrar de fornecedor e atualizar 27
23. Resolver erros de instalação 28
24. Entrega e referências 29

1. Escolher o percurso certo

O Food Truck Live precisa de dois sites Next.js, uma API Node.js, PostgreSQL, armazenamento persistente de imagens e email para confirmação/recuperação de contas.

Percurso	Componentes e indicação
A - Vercel e Supabase	Dois sites e API no Vercel; PostgreSQL e imagens no Supabase; email Brevo. Modelo da instalação de referência.
B - Docker e Cloudflare DNS	Serviços num servidor Docker; PostgreSQL e disco em volumes; Cloudflare para domínio/proteção; Brevo ou SMTP. Mais controlo e manutenção.
C - Workers e API externa	Sites Cloudflare Workers com OpenNext; API num alojamento Node.js/Vercel; PostgreSQL externo; imagens Supabase/S3/R2. Percurso avançado.
D - Node.js genérico	Sites e API em processos Node.js; PostgreSQL gerido ou próprio; proxy HTTPS e armazenamento escolhido.

Cloudflare e Supabase não são substitutos diretos: Cloudflare pode alojar os sites e proteger domínios; Supabase fornece PostgreSQL e armazenamento. Podem ser usados em conjunto. Carregar ficheiros num alojamento estático/PHP ou criar só uma base não instala esta aplicação.

As capturas desta edição foram recolhidas em 3 de novembro de 2026. Mostram a aplicação real ou exemplos publicados pelos fornecedores, identificados nas legendas. O aspeto pode variar com a versão e o tamanho do ecrã.

2. Compatibilidade e limites

Requisitos de referência: Node.js 24, npm com workspaces e PostgreSQL. A instalação de referência usa PostgreSQL 17; o ambiente local também usa 18. Noutras versões, valide migrações e testes numa base separada.

Componente	Compatibilidade preparada
Sites	Node.js/Next.js, Docker ou Vercel; modelos opcionais OpenNext para Workers.
API	Node.js/Fastify em servidor ou contentor; função HTTP própria no Vercel.
Base	PostgreSQL com transações, JSONB e permissões necessárias.
Imagens	Disco persistente, bucket privado Supabase ou API S3, incluindo R2.
Email	API Brevo ou SMTP externo.
Sistema	Windows, macOS e Linux com Node.js; Docker requer contentores Linux suportados pelo anfitrião.

MySQL, MariaDB, SQL Server, SQLite e Cloudflare D1 não são substituições diretas. A versão atual usa funcionalidades PostgreSQL. Não há promessa de instalar sem adaptação em todos os motores ou planos de alojamento.

O atalho/PWA depende do navegador; não é um pacote nativo e não remove a necessidade de ligação ao servidor.

3. Conteúdo do pacote e dados privados

O pacote contém código, dependências bloqueadas, migrações, imagens públicas, recursos UPLOADS, exemplos, Docker, modelos de publicação e os manuais. Não inclui base operacional, contas/clientes, encomendas, passwords, chaves, sessões, ficheiros .env privados, node_modules ou caches. Uma instalação nova começa vazia.

UPLOADS contém recursos para inserção manual. Não é o armazenamento operacional da API. Fazer backup dessa pasta não substitui copiar data/uploads ou os objetos do bucket.

Caminho	Função
apps/customer	Site dos clientes, porta habitual 3000.
apps/management	Gestão, porta habitual 3001.
services/api	API PostgreSQL, porta habitual 4000.
services/api/migrations	Evolução da estrutura da base.
.env.install.example	Modelo da configuração portátil.
compose.install.yaml	Instalação Docker com PostgreSQL e HTTPS.
deployment/cloudflare	Modelos opcionais Workers.
docs/manuals	Fontes e PDFs dos manuais.

Há código da versão antiga no repositório. Este manual usa a arquitetura PostgreSQL atual; não execute scripts legacy SQLite para a instalar.

4. Obter o código por GitHub ou ZIP

Instale Node.js 24 e Git dos sites oficiais. Confirme node --version, npm --version e git --version no terminal.

Faça Fork do repositório ou envie o código para um repositório seu. Use um repositório privado se não quer divulgar conteúdos. Nunca envie dados de clientes ou chaves.

```
git clone https://github.com/aimiguelazio01/foodtruck.git
cd foodtruck
npm ci --include=dev
npm run install:init
```

Com ZIP, extraia para uma pasta sua, abra o terminal nessa pasta e execute npm ci e install:init. Não execute dentro do ZIP. O nome da pasta pode variar.

install:init cria .env.install apenas se ainda não existir. Edite esse ficheiro com os seus valores e guarde. Não o publique. Depois execute npm run install:check. O diagnóstico de configuração não cria base, compra serviços ou publica a aplicação.

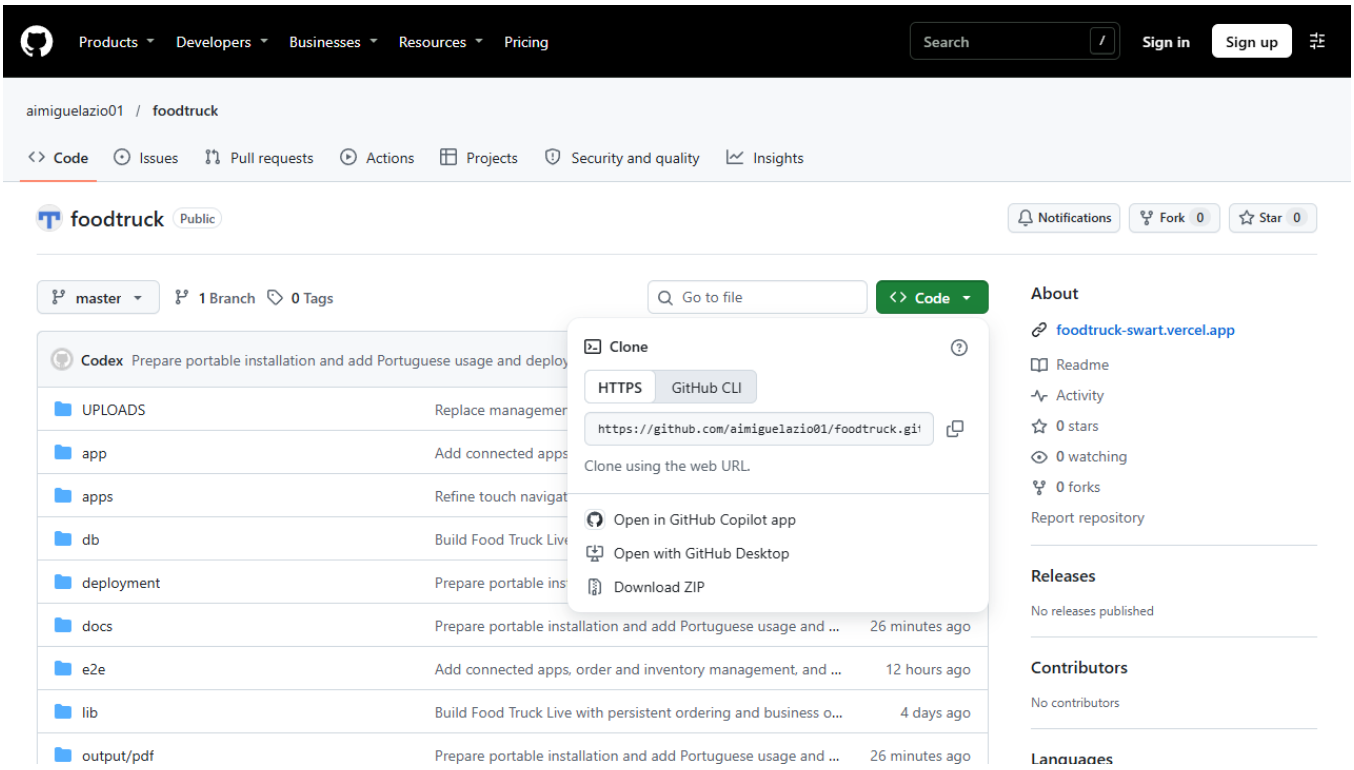


Figura 1. Repositório real no GitHub, menu Code aberto: copiar a ligação ou escolher Download ZIP.

Fonte: https://github.com/aimiguelazio01/foodtruck

5. Variáveis de ambiente

No alojamento, as variáveis do painel substituem o ficheiro local. Configure Production e Preview separadamente. Não ligue um preview de teste a dados de produção por engano.

Variável	Destino e função
DATABASE_URL	API/worker: PostgreSQL de execução.
DATABASE_MIGRATION_URL	Instalação: ligação direta/session para migração e backup.
DATABASE_CA_CERT_PEM	Servidor: conteúdo do certificado oficial da base.
DATABASE_CA_CERT_FILE	Node/Docker: caminho de certificado existente no servidor.
CUSTOMER_APP_URL	API: URL público final dos clientes.
MANAGEMENT_APP_URL	API: URL público final da gestão.
CORS_ORIGINS	API: duas origens separadas por vírgula, sem barra final.
INTERNAL_API_URL	Sites, no build e runtime: URL da API.
API_UPDATES_MODE	API: poll para transaction pooler ou sem streams; stream exige ligação adequada e worker.
DATA_DIRECTORY	API/worker: diretório persistente se usar disco.
DEMO_MODE	Usar 0 em produção; não cria dados de demonstração.

Email e storage são descritos nos capítulos seguintes. SUPABASE_SERVICE_ROLE_KEY, credenciais S3/R2 e Brevo só ficam na API. Não use prefixo NEXT_PUBLIC_ em segredos. A chave publishable/anon Supabase não substitui a ligação PostgreSQL ou chave de servidor. Não desative TLS para contornar erros.

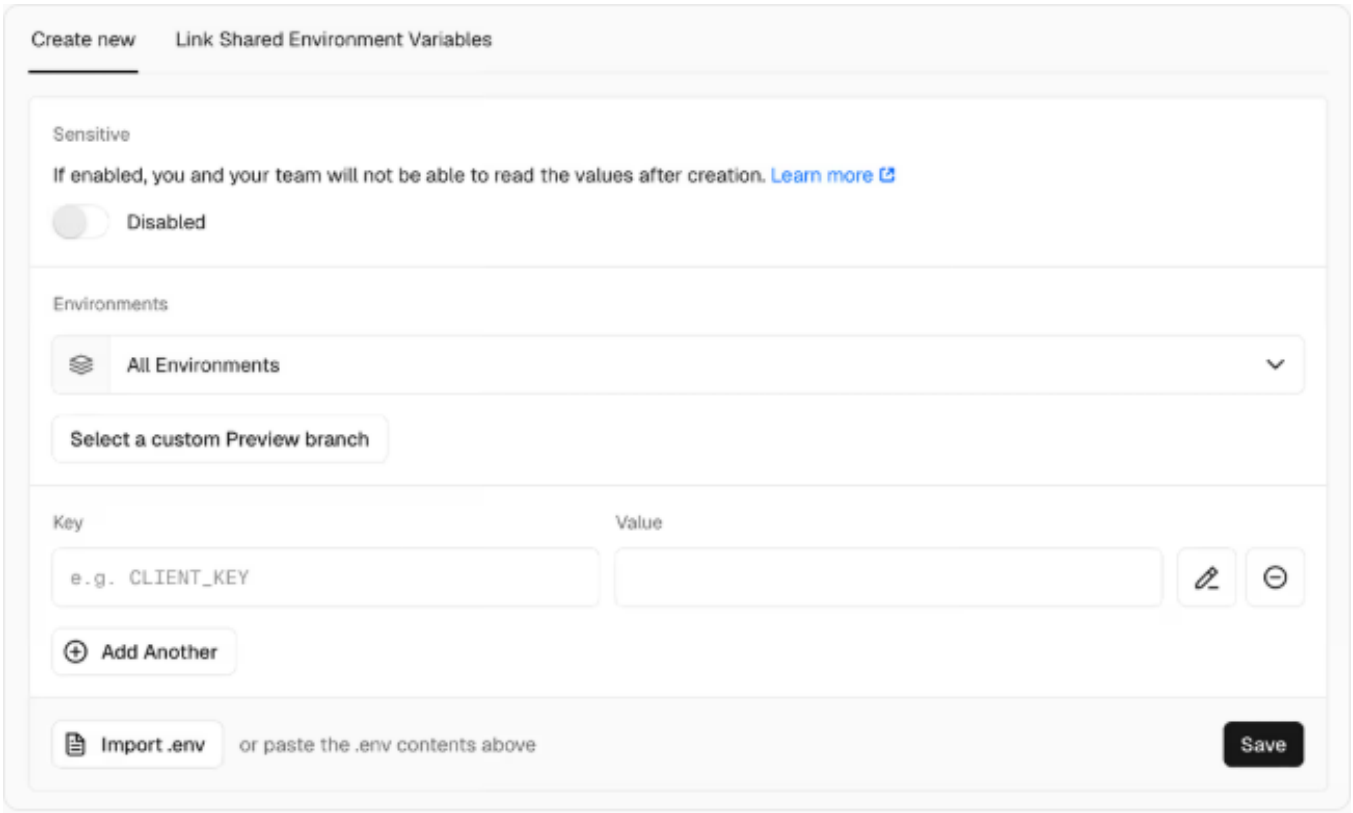


Figura 2. Interface de variáveis, publicada na documentação oficial Vercel. Os valores da imagem são exemplos do fornecedor; use as variáveis deste manual.

Fonte: https://vercel.com/docs/environment-variables

6. Criar PostgreSQL no Supabase

1. Entre em https://supabase.com/dashboard e crie um projeto na sua organização.

2. Escolha região próxima da API, nome e password forte da base; guarde-a num gestor de passwords. Aguarde o projeto estar disponível.

3. Em Connect, identifique Direct, Session pooler e Transaction pooler. Copie os endereços reais; não invente o hostname.

4. Use Direct ou Session para instalar/migrar. Para API serverless, use Transaction, habitualmente na porta 6543.

5. No SQL Editor, use deployment/postgres-bootstrap.sql.example, substituindo o segredo. O modelo é para base nova, não para reparar uma existente.

O modelo cria foodtruck_app e o esquema foodtruck, define o search_path e retira acesso público. No Supabase, retire também acesso a anon/authenticated conforme o modelo; não exponha foodtruck na Data API.

No pooler, o utilizador costuma incluir a referência do projeto: foodtruck_app.REFERENCIA. Use o formato do painel para esse papel. Caracteres especiais na password precisam de codificação URL.

Guarde as duas ligações em .env.install. Obtenha o certificado oficial indicado pelo Supabase; use DATABASE_CA_CERT_FILE localmente ou DATABASE_CA_CERT_PEM no Vercel. Valide TLS.

As contas de clientes estão nas tabelas da API; a versão atual não usa Supabase Auth. Alterar os modelos de email do Supabase Auth não configura o envio desta aplicação [9].

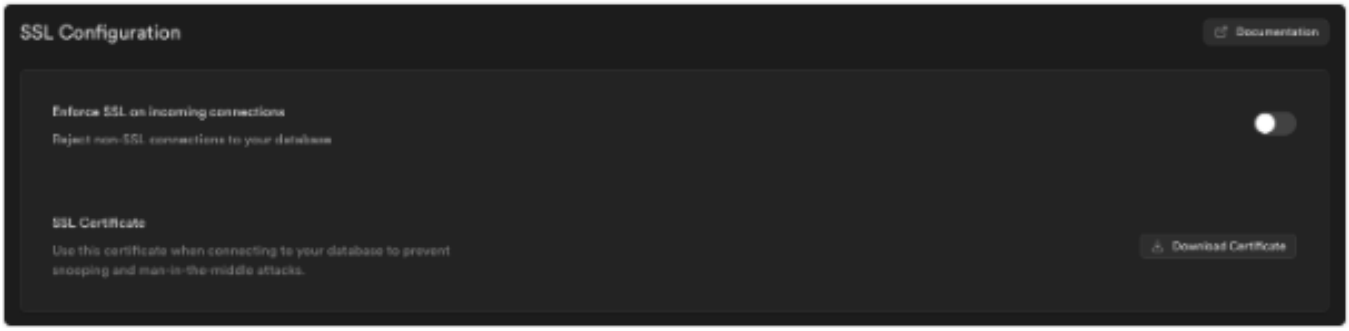


Figura 3. Painel SSL ilustrado na documentação oficial Supabase: localizar Download Certificate. Esta não é uma captura da sua conta.

Fonte: https://supabase.com/docs/guides/database/connecting-to-postgres

7. Outro fornecedor PostgreSQL

Pode usar PostgreSQL próprio ou gerido, acessível por rede privada ou ligação autorizada. Escolha região, espaço, backups e limite de ligações.

Crie uma base exclusiva e um utilizador autorizado a criar/alterar as tabelas. Se usar esquema dedicado, configure o search_path do papel. O modelo SQL contém GRANT para o administrador postgres usado no Supabase; ajuste essa linha se o administrador do seu fornecedor for outro. Não execute comandos anon/authenticated se esses papéis não existem.

```
postgresql://utilizador:SEGREDO_CODIFICADO@host:5432/base
```

Para API persistente com LISTEN/NOTIFY, use Direct/Session e worker. Com Transaction, use API_UPDATES_MODE=poll. Neste modo há manutenção em pedidos; um worker dedicado permite manutenção sem depender de visitas.

Não abra PostgreSQL a toda a internet sem necessidade. Use rede privada ou autorize os acessos necessários. Use pg_dump/pg_restore compatíveis e ensaie restauros entre versões. Portabilidade não dispensa testar permissões, TLS e capacidade.

8. Migrações e primeiro administrador

Depois de configurar uma base nova:

```
npm run install:check
npm run db:migrate
npm run install:doctor
npm run db:bootstrap
```

db:migrate aplica SQL pendente e regista migrações. Usa DATABASE_MIGRATION_URL se definida; requer ligação estável Direct/Session. install:doctor confirma ligação, esquema e registo de migrações.

O bootstrap pede email, nome da organização e password oculta num terminal interativo. Aceita 12 a 256 caracteres e guarda hash, sem imprimir o segredo. Para automação, use BOOTSTRAP_EMAIL, BOOTSTRAP_ORGANIZATION e BOOTSTRAP_PASSWORD como variáveis privadas temporárias; retire a password do ambiente quando terminar, sem a escrever no Git ou histórico de comandos.

Só funciona se não existem utilizadores, organizações ou negócios. Repetir recusa criar/modificar contas. Não use numa instalação já utilizada. Depois do primeiro login, abra Rede > Negócios e crie a primeira carrinha. Não são copiados dados da instalação de referência.

9. Armazenamento das imagens

Opção	Configuração
Disco	DATA_DIRECTORY num volume persistente; incluir nos backups.
Supabase privado	SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_STORAGE_BUCKET na API.
S3/R2	S3_BUCKET, S3_ENDPOINT se aplicável, AWS_REGION e credenciais do bucket.

A prioridade é Supabase quando configurado, depois S3 e finalmente disco. Ao mudar, retire configurações antigas. Copie os objetos mantendo as chaves; criar outro bucket não transfere imagens.

No Supabase Storage, crie o bucket privado foodtruck-media. Os objetos usam uploads/ e nomes UUID.webp. Não publique a chave de servidor no frontend.

Disco temporário não é persistente. No Vercel, configure storage externo antes de permitir uploads; DATA_DIRECTORY=/tmp/foodtruck é temporário. Teste uma imagem na gestão e no menu público, incluindo depois de nova publicação.

10. Ativar email com Brevo ou SMTP

1. Crie uma conta Brevo e confirme o email de acesso.

2. Adicione e verifique o remetente. Com domínio próprio, configure a autenticação DNS indicada pelo fornecedor.

3. Em SMTP & API > API Keys, crie uma chave API da instalação. Não use uma chave SMTP em BREVO_API_KEY.

4. Guarde BREVO_API_KEY como segredo apenas na API.

5. Defina MAIL_FROM no formato Food Truck Live <confirmacoes@seu-dominio.pt>, com remetente validado.

6. Publique novamente a API e teste registo, receção, confirmação, login e recuperação.

Em alternativa, configure SMTP_URL e MAIL_FROM. Se BREVO_API_KEY existir, tem prioridade; retire-a para passar a SMTP.

«Conta existente» não prova que o email chegou. Use «Reenviar confirmação» com o endereço correto. Veja nos logs Brevo se a mensagem foi aceite, entregue ou devolvida; verifique spam. Um outbox local não envia email. Não confirme contas diretamente na base para contornar configuração incompleta.

O plano gratuito indica 300 envios/dia, partilhados com os restantes envios da conta, sem acumulação. Confirme disponibilidade transacional e limites na sua conta [4].

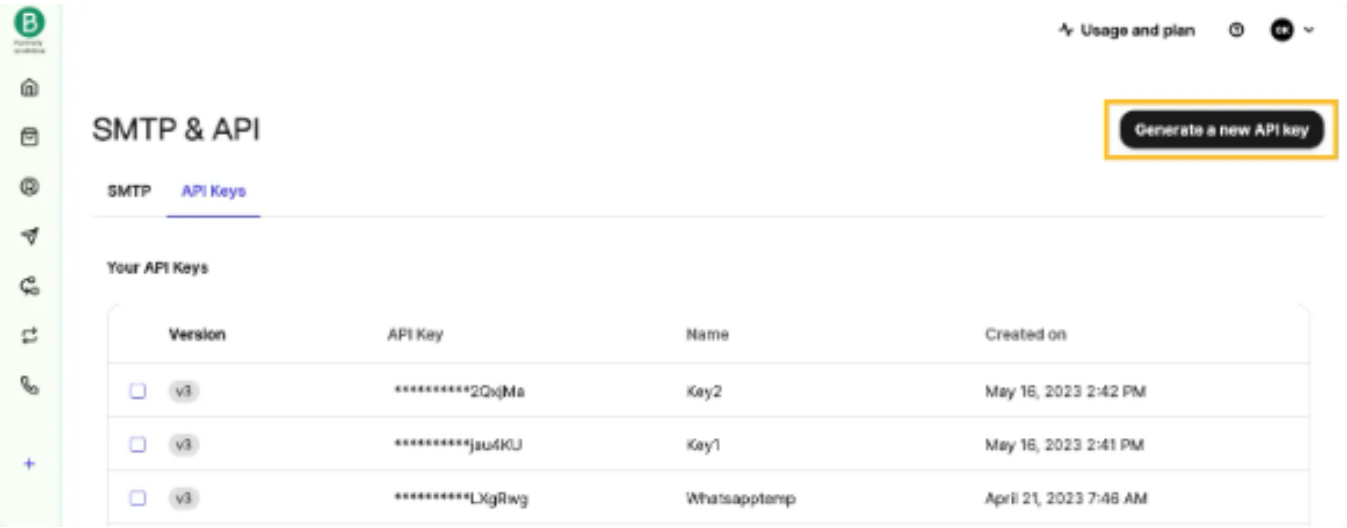


Figura 4. Ecrã SMTP & API do guia oficial Brevo: escolher API Keys e Generate a new API key. Chaves existentes aparecem mascaradas.

Fonte: https://developers.brevo.com/docs/api-key-authentication

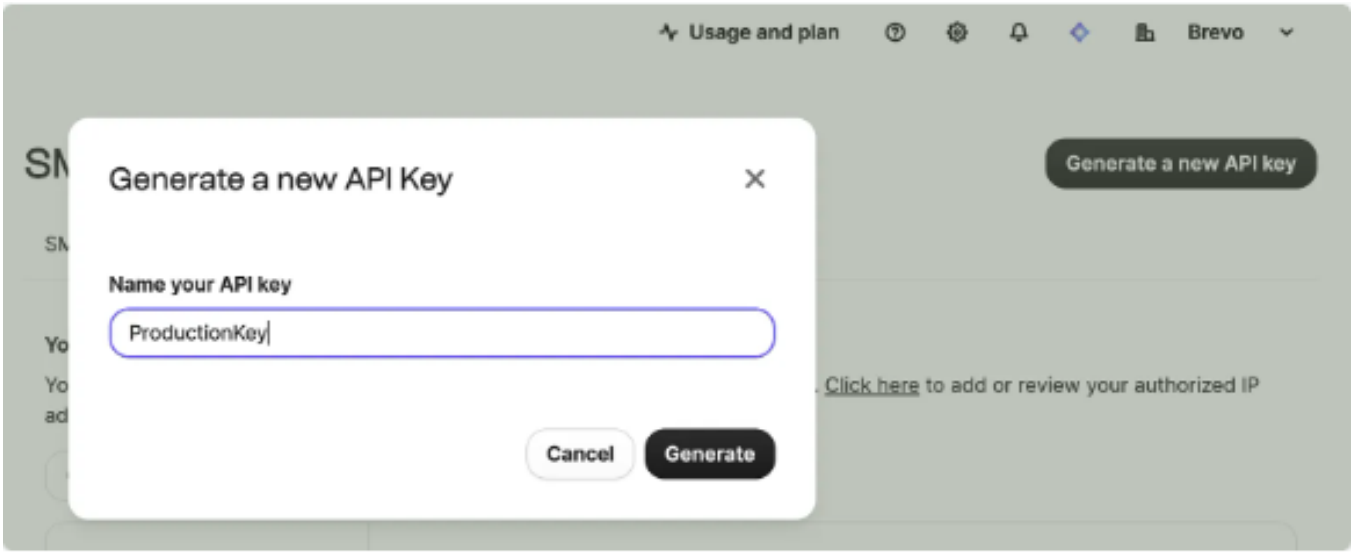


Figura 5. Diálogo de nome da chave no guia oficial Brevo. Crie uma chave própria; nunca a publique nos manuais ou no GitHub.

Fonte: https://developers.brevo.com/docs/api-key-authentication

11. Percurso A: API no Vercel

Ligue o seu repositório GitHub ao Vercel. Crie três projetos do mesmo repositório. Primeiro publique a API.

Definição da API	Valor
Root Directory	services/api
Framework Preset	Other; respeitar o vercel.json incluído.
Node.js	24.x
Handler	api/index.ts, definido no vercel.json.
Output Directory	Sem override Next.js; não usar .next na API.

Nas variáveis Production, configure DATABASE_URL (Transaction), DATABASE_CA_CERT_PEM, SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_STORAGE_BUCKET, BREVO_API_KEY, MAIL_FROM e DEMO_MODE=0. Use DATA_DIRECTORY=/tmp/foodtruck. A API deteta Vercel e usa atualização periódica.

Defina CUSTOMER_APP_URL, MANAGEMENT_APP_URL e CORS_ORIGINS com os URLs finais dos sites. Se ainda não existem, crie/reserve os projetos e complete estes valores antes dos testes. Nunca deixe example.com.

Aplique migrações e bootstrap da sua máquina com .env.install; a função Vercel não instala a base automaticamente. Conclua Deploy e confirme HTTP 200 em https://SUA-API/api/v1/health. A raiz da API não é uma página de login.

As sessões ficam no PostgreSQL. Password da base e segredos ficam só no projeto da API.

12. Percurso A: sites no Vercel

Importe o mesmo GitHub mais duas vezes como projetos distintos.

Definição	Clientes	Gestão
Root Directory	apps/customer	apps/management
Framework	Next.js	Next.js
Build Command	npm run build	npm run build
Output Directory	.next	.next
Node.js	24.x	24.x
INTERNAL_API_URL	URL HTTPS da API	URL HTTPS da API

Mantenha package-lock.json na raiz do monorepo. Permita acesso a ficheiros partilhados fora do Root Directory se essa opção for apresentada [6].

INTERNAL_API_URL tem de estar no build e runtime. Depois de mudar essa variável, publique novamente os sites: /api/v1 e /uploads usam esse destino.

Quando souber os dois URLs finais, atualize CUSTOMER_APP_URL, MANAGEMENT_APP_URL e CORS_ORIGINS na API e publique-a novamente. As origens são separadas por vírgula, sem caminho ou barra final.

Entre na gestão com a conta do bootstrap, crie uma carrinha e confirme «Ver página de encomendas». Teste cliente, email, imagem, evento e encomenda antes de divulgar.

NEXT_NO_ROUTES_MANIFEST costuma indicar raiz/framework/output incorretos ou build falhado. Veja a publicação nova, não um link antigo que continua falhado. A raiz geral do repositório não é o Root Directory dos sites.

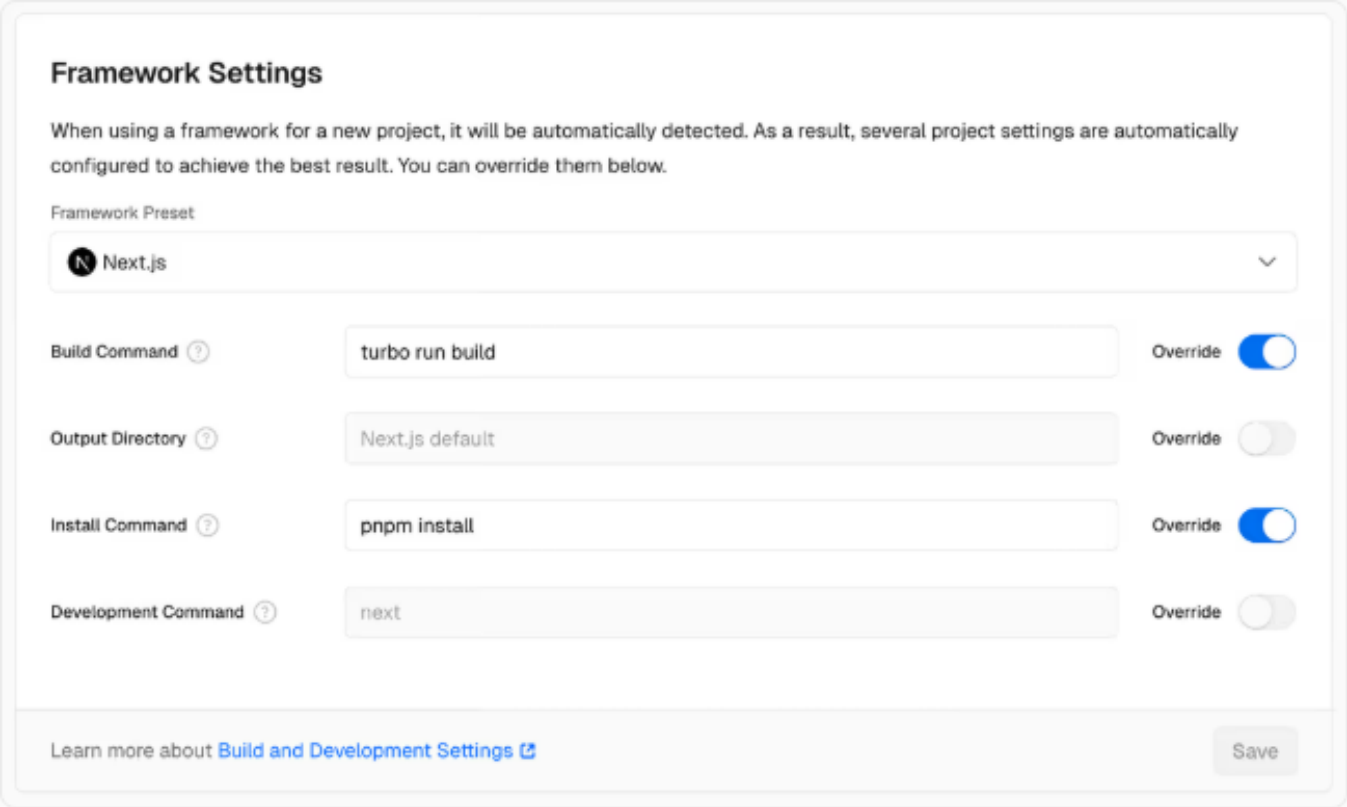


Figura 6. Framework Settings, imagem da documentação oficial Vercel. Para esta aplicação, use npm run build e as raízes/output da tabela anterior; não copie os comandos de exemplo da imagem.

Fonte: https://vercel.com/docs/builds/configure-a-build

13. Percurso B: servidor Docker

Requer servidor com Docker Engine e Compose v2, espaço para base/imagens e dois domínios seus. Docker Desktop em Windows/macOS permite ensaios; produção pode usar Linux.

Crie .env.install, configure URLs HTTPS, CORS e domínios sem https://. Use POSTGRES_PASSWORD aleatória hexadecimal para evitar problemas na interpolação do URL. A base do Compose é independente de bases locais já existentes.

```
docker compose --env-file .env.install -f compose.install.yaml config --quiet
docker compose --env-file .env.install -f compose.install.yaml up -d --build
```

O Compose cria PostgreSQL 17, API, worker, sites e Caddy. A API aplica migrações no arranque; os volumes conservam base, imagens e certificados. O gateway recebe 80/443.

Quando os serviços estiverem saudáveis, crie o administrador:

```
docker compose --env-file .env.install -f compose.install.yaml run --rm api node scripts/bootstrap-network.mjs
```

O terminal pede credenciais; não escreva a password na linha de comando. Consulte estado e logs com os mesmos parâmetros --env-file e -f, seguidos de ps ou logs --tail 100 api.

Não exponha PostgreSQL no router. Não execute down -v com dados: essa opção elimina volumes. A receita usa base interna; só mudar DATABASE_URL no ficheiro não altera o override do Compose. Para base externa, adapte serviços/dependências ou use Node.js genérico.

14. Cloudflare: DNS e proteção do Docker

Os serviços continuam no seu servidor. Cloudflare gere DNS e, se ativado, proxy/proteção; não é necessário converter a API em Worker.

1. Adicione o domínio ao Cloudflare e conclua a configuração DNS.

2. Crie A/AAAA para os subdomínios clientes/gestão, apontando ao IP público do servidor.

3. Confirme portas 80/443 livres e acessíveis.

4. Para emitir os certificados Caddy, pode começar em DNS only, aguardar a emissão e testar HTTPS direto.

5. Se ativar o proxy, use SSL/TLS Full (strict), com certificado válido na origem. Evite Flexible.

6. Não aplique cache geral a /api/v1, login, conta ou encomendas.

7. Teste login, disponibilidade, imagem e encomenda no domínio final.

DNS Free não inclui uma máquina Linux ou PostgreSQL gratuito. O servidor tem os seus custos e manutenção. Cloudflare Pro para o site e Workers Paid são produtos diferentes.

Esta é a opção mais direta para manter a aplicação completa em Docker e usar Cloudflare no domínio.

15. Percurso C: clientes em Workers

Foram incluídos modelos e comando de preparação. Valide no runtime Workers antes de produção: não foi ativada conta nem feita publicação Cloudflare nesta preparação.

Mantenha a API Fastify em Vercel ou outro Node.js. Os sites Workers acedem por HTTPS. Dependências e tratamento de imagem da API não são convertidos por copiar services/api para um Worker.

OpenNext declara suporte a Next.js 16 [7]. A documentação Cloudflare pode apresentar outras ferramentas; a opção fornecida conserva Next.js. Confirme as versões e execute preview. Se as ferramentas tiverem limitações no Windows, use WSL2/Linux.

Em PowerShell, configure também o URL no ambiente do build:

```
$env:INTERNAL_API_URL='https://SUA-API.example.com'
npm run cloudflare:prepare -- customer https://SUA-API.example.com
npm run cf:build --workspace=food-truck-customer
npm run cf:preview --workspace=food-truck-customer
```

Em Bash, comece com export INTERNAL_API_URL='https://SUA-API.example.com'. O prepare instala @opennextjs/cloudflare e wrangler no workspace, copia configurações e acrescenta comandos cf. Altera package.json/lock da sua cópia; não publica e não substitui wrangler.json existente.

Depois de o preview passar:

```
npx wrangler login
npm run cf:deploy --workspace=food-truck-customer
```

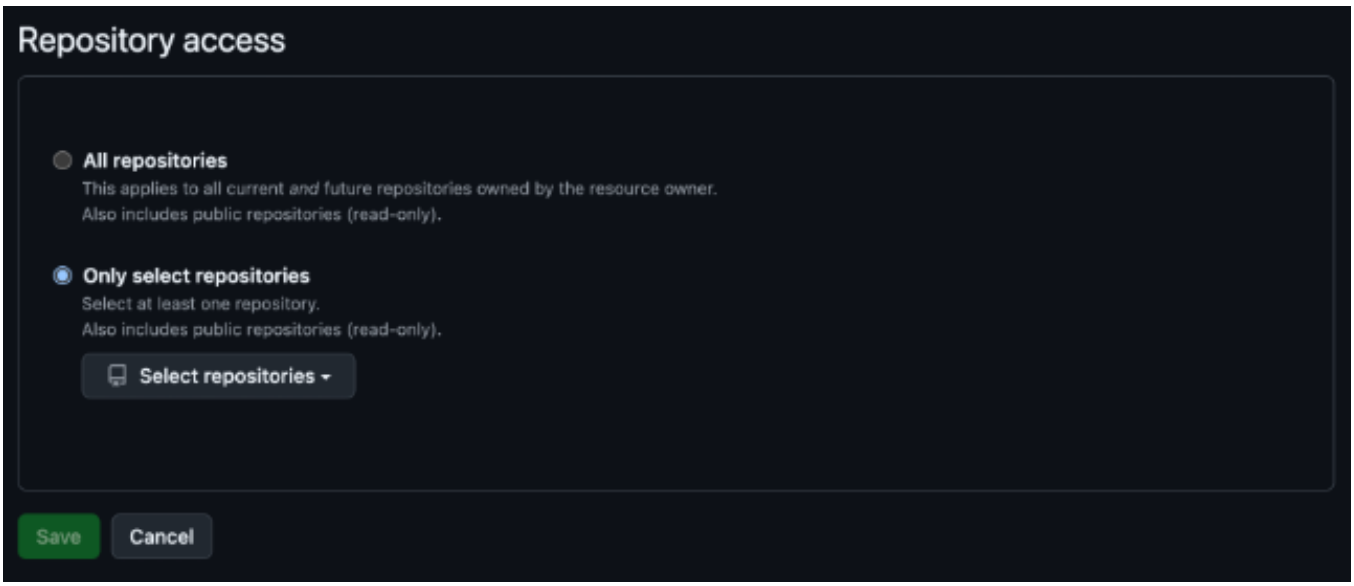


Figura 7. Autorização de repositórios GitHub, ilustrada no guia oficial Cloudflare Workers. Autorize apenas o repositório necessário.

Fonte: https://developers.cloudflare.com/workers/ci-cd/builds/git-integration/github-integration/

16. Workers: gestão e GitHub

Repita para o segundo site:

```
npm run cloudflare:prepare -- management https://SUA-API.example.com
npm run cf:build --workspace=food-truck-management
npm run cf:preview --workspace=food-truck-management
npm run cf:deploy --workspace=food-truck-management
```

Cada site tem Worker/nome próprios, saída .open-next e flags nodejs_compat/global_fetch_strictly_public. WORKER_SELF_REFERENCE deve usar o mesmo nome. Nunca coloque credenciais de base/Brevo nos sites.

Para GitHub em Workers Builds, ligue o repositório e use raiz do monorepo, instalação npm ci, variável de build INTERNAL_API_URL, build npm run cf:build --workspace=food-truck-customer e deploy npm run cf:deploy --workspace=food-truck-customer. Repita para o workspace da gestão. Envie primeiro as alterações do prepare para o seu GitHub.

Configure workers.dev ou domínios personalizados. Atualize URLs e CORS na API externa, publique-a e teste sessões através dos sites finais.

Não escolha HTML estático ou next-on-pages como substituição automática deste percurso dinâmico. Se preview falhar, corrija antes de publicar; Node/Docker continua disponível. Workers Free pode não ter CPU suficiente para páginas dinâmicas: avalie com carga real.

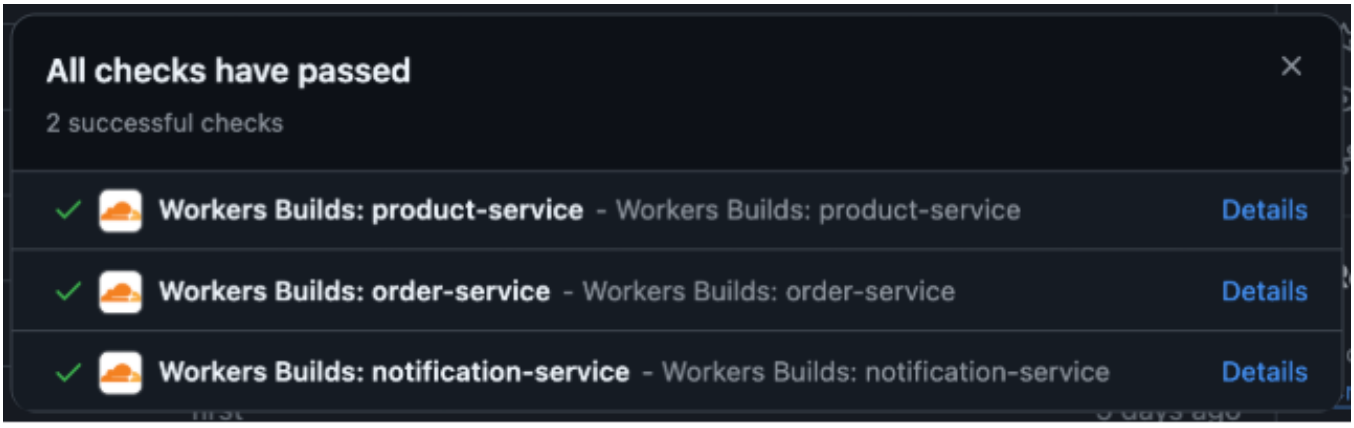


Figura 8. Exemplo de verificações do build publicado no guia Cloudflare Workers. Confirme os resultados antes de usar uma publicação.

Fonte: https://developers.cloudflare.com/workers/ci-cd/builds/git-integration/github-integration/

17. R2 para imagens, mantendo PostgreSQL

R2 é armazenamento de objetos, não a base relacional. Pode substituir o bucket de imagens e manter PostgreSQL no fornecedor escolhido.

1. Crie bucket R2 privado e credenciais S3 limitadas a leitura/escrita desse bucket.

2. Na API, defina S3_BUCKET, S3_ENDPOINT com o endpoint S3 do painel, AWS_REGION=auto, AWS_ACCESS_KEY_ID e AWS_SECRET_ACCESS_KEY.

3. Retire SUPABASE_URL/SUPABASE_SERVICE_ROLE_KEY da configuração se pretende que R2 seja o destino ativo.

4. Copie objetos existentes preservando uploads/NOME.webp; confira contagem e amostras.

5. Publique a API, carregue uma imagem e confirme nos sites.

O bucket pode ser privado: a API serve /uploads com as credenciais do servidor. Nunca coloque essas credenciais no navegador. Um dump PostgreSQL contém referências às imagens, não os ficheiros. O ZIP também não exporta automaticamente o bucket operacional.

R2 tem franquias e cobrança por armazenamento/operações; consulte preços de Standard e restantes classes [8]. Gratuito não significa ilimitado ou dispensa de método de pagamento.

18. Percurso D: Node.js genérico

Num fornecedor Node.js 24, instale npm ci --include=dev e configure base, storage e variáveis. Em serviços separados, use caminhos absolutos para DATA_DIRECTORY e DATABASE_CA_CERT_FILE. Com .env.install:

```
npm run install:check
npm run db:migrate
npm run db:bootstrap
npm run build:install
npm start
```

npm start supervisiona API, worker e sites num servidor persistente. Não é o comando para Vercel Functions/Workers. Use um gestor de serviços para reinício e logs.

Separadamente: compile a API com npm run build --workspace=@foodtruck/api e execute npm run start --workspace=@foodtruck/api; execute npm run worker --workspace=@foodtruck/api num serviço próprio. Compile cada site com INTERNAL_API_URL no ambiente e execute o seu start com a porta atribuída.

Use proxy HTTPS à frente dos sites. Cookies Secure em produção exigem HTTPS; HTTP simples não é teste válido de login. Caddyfile.network exemplifica dois domínios e as rotas /api/v1 e /uploads. Nginx ou outro proxy pode fazer o mesmo, preservando cookies, métodos e corpos HTTP. Não exponha portas internas sem necessidade.

Streams exigem ligação adequada e proxy sem buffering; em alternativa use API_UPDATES_MODE=poll. Só ter o nome de um fornecedor não garante compatibilidade: confirme processos Node, versões, rede, disco e limites.

19. Gratuito ou pago

Limites consultados em 3 de novembro de 2026. USD sem impostos, câmbio, domínios e excedentes. Confirme sempre preço/condições antes de contratar.

Serviço	Gratuito	Pago e consequência
Supabase	500 MB base, 1 GB ficheiros; pode pausar após semana inativa.	Pro desde 25 USD/mês, mais capacidade e backups diários retidos 7 dias; podem acrescer consumos/projetos. [1]
Vercel	Hobby: uso pessoal não comercial; útil para ensaios e GitHub.	Pro para trabalho profissional/comercial; preço anunciado desde 20 USD/mês, com condições por lugar/consumo. Confirmar orçamento. [2][3]
Workers	100 000 pedidos/dia e 10 ms CPU/pedido, partilhados na conta.	Workers Paid desde 5 USD/mês, com franquias/excedentes e mais CPU. Não inclui PostgreSQL. [5]
Brevo	300 emails/dia; pode limitar confirmações.	Mais volume segundo oferta contratada; não garante entrega na caixa principal. [4]
Docker	Software pode ser gratuito; máquina, disco e rede têm custos.	VPS/servidor e backups ao preço do fornecedor; controlo com responsabilidade de manutenção.

Grátis favorece ensaios de baixo tráfego, mas pode ter pausas, quotas e menos apoio. Pago pode aumentar capacidade, continuidade, backups e suporte; não corrige configurações erradas.

Some os três componentes web/API, base, imagens, email, domínio e backups. Use alertas e limites de despesa. Outros projetos da mesma conta podem consumir as mesmas quotas. Não escolha o conjunto apenas pela frase «site grátis».

20. Testar antes de abrir ao público

Use dados seus e uma carrinha de teste, em ambiente separado quando possível.

1. API health: HTTP 200; doctor encontra base e migrações.

2. Login da gestão no domínio final e utilizador correto.

3. Criar negócio e verificar slug/menu público.

4. Fotografias e logo persistem depois de nova publicação.

5. Ingrediente, produto e receita apresentam stock/disponibilidade corretos.

6. Evento publicado tem hora/coordenadas e aparece atual ou próxima paragem.

7. Cliente recebe confirmação, confirma, entra e recupera password.

8. Encomenda mostra número, acompanhamento e associação à conta.

9. Estados, recolha e pagamento funcionam conforme a operação real.

10. Atual/Touch funciona nas quatro áreas e após recarregar.

11. Excel descarrega e abre.

12. Backup é restaurado com sucesso numa base separada.

Vercel/Supabase/Brevo são o percurso usado na referência. O bootstrap novo foi testado em esquema isolado e a API compilada. Não há Docker disponível neste computador e não foi feita publicação Workers nesta preparação; valide esses destinos antes de produção.

21. Backup e restauro

Instale pg_dump/pg_restore compatíveis. Use DATABASE_MIGRATION_URL Direct/Session, DATABASE_CA_CERT_FILE se necessário e BACKUP_SCHEMA=foodtruck no Supabase para copiar só esse esquema.

```
npm run backup
```

O comando cria dump em data/backups e copia imagens locais. Com Supabase/S3/R2, avisa que objetos externos não estão incluídos: copie-os separadamente. Preserve também recursos UPLOADS desejados.

Guarde cópias cifradas fora do servidor, com retenção e acesso limitado. Teste o restauro: um ficheiro existente não garante recuperação. Backups de base do fornecedor não substituem a estratégia para imagens e configuração.

Para restaurar, crie destino vazio e papel/esquema apropriados. Execute pg_restore --no-owner --no-acl no dump, com PGHOST, PGPORT, PGUSER, PGPASSWORD, PGDATABASE e TLS definidos privadamente. Não escreva passwords em comandos partilhados. O esquema do dump tem de corresponder ao search_path do papel.

Restaure também os objetos com as mesmas chaves. Aponte uma instalação de ensaio ao destino, execute diagnóstico e confirme negócios, produtos, imagens e encomendas. Planeie só depois a troca do serviço ativo.

Não use --clean, DROP ou remoção de volumes sobre produção sem plano de recuperação e autorização concreta. Instalação e restauro são operações diferentes.

22. Migrar de fornecedor e atualizar

Prepare o destino mantendo a origem. Instale inicialmente a mesma versão do código.

1. Faça backup completo e restauro de ensaio.

2. Configure segredos no destino sem os enviar ao GitHub.

3. Copie PostgreSQL, preservando esquema/search_path.

4. Copie imagens, conferindo nomes e contagens.

5. Reserve janela sem novas escritas para cópia final; não deixe duas bases independentes a receber encomendas.

6. Atualize domínios, CORS e URLs de email; recompile os sites se a API mudou.

7. Teste tudo, acompanhe logs e preserve a origem até validar a troca.

Numa atualização, faça backup e ensaie em staging. Reveja alterações; execute git pull, npm ci, db:migrate e build:install, e reinicie conforme o alojamento. No Vercel, o push pode publicar automaticamente, mas migrar a base é etapa separada.

Não repita db:bootstrap. Rollback de código não desfaz migração de base; avalie a compatibilidade antes de voltar a uma versão.

23. Resolver erros de instalação

Sintoma	Verificação
NEXT_NO_ROUTES_MANIFEST	Raiz dos sites, preset Next.js, output .next e erro anterior do build.
Página não carrega	API/rewrite para localhost, base pausada, variável alterada sem rebuild.
Origin not allowed	CORS_ORIGINS com os domínios finais; publicar API.
Sessão desaparece	HTTPS, cookie, proxy e domínio; produção em HTTP não serve.
Tabela inexistente	Migração ausente ou search_path errado.
Too many clients	Pool/limite de ligações; serverless Transaction + poll, sem LISTEN permanente.
Certificado inválido	Certificado oficial, hostname e TLS; não desativar validação.
Imagem desaparece	Disco efémero ou objetos não copiados.
Email não chega	Chave API versus SMTP, remetente, logs, limite e spam.
Evento sem pin	Publicação, data/hora e coordenadas guardadas.
Workers preview falha	Versões, adaptador, flags e limites; corrigir antes do deploy.

Comece por install:check, install:doctor e logs do componente que falha. Registe hora e ação, sem partilhar segredos.

24. Entrega e referências

Entregue URLs, conta individual do administrador, repositório da organização, local privado dos segredos, fornecedores/planos, rotina de backup/restauro e suporte. Registe versão/commit, região, data e orçamento. A conta pessoal do instalador não deve ser o único proprietário.

Dados de clientes e encomendas são privados; controle acessos a dumps/Excel e retenção. Pagamento online e faturação fiscal exigem integrações adicionais, não ativadas pelo instalador.

Fontes oficiais consultadas:

[1] Planos Supabase: https://supabase.com/pricing

[2] Vercel Hobby: https://vercel.com/docs/plans/hobby

[3] Preços Vercel: https://vercel.com/pricing

[4] Brevo Free: https://help.brevo.com/hc/en-us/articles/208580669-FAQs-What-are-the-limits-of-the-Free-plan

[5] Workers: https://developers.cloudflare.com/workers/platform/pricing/ e https://developers.cloudflare.com/workers/platform/limits/

[6] Monorepos Vercel: https://vercel.com/docs/monorepos

[7] OpenNext: https://opennext.js.org/cloudflare/get-started e https://opennext.js.org/cloudflare

[8] R2: https://developers.cloudflare.com/r2/pricing/

[9] Ligações PostgreSQL: https://supabase.com/docs/guides/database/connecting-to-postgres

[10] Cloudflare Next.js e D1: https://developers.cloudflare.com/workers/framework-guides/web-apps/nextjs/ e https://developers.cloudflare.com/d1/

Reconfirme preços na contratação. Os exemplos não prometem custo zero ou capacidade ilimitada.